

如何使用 App Engine Blobstore (第 15 個模組)

來源：<https://codelabs.developers.google.com/codelabs/cloud-gae-python-migrate-15-blobstore?hl=zh-tw>

步驟數：7

瞭解如何將 Blob 使用情形新增至簡易的 Python 2 App Engine 應用程式

目錄

1. [1. 總覽](#)
2. [2. 背景](#)
3. [3. 設定/準備工作](#)
4. [4. 更新設定檔](#)
5. [5. 修改應用程式檔案](#)
6. [6. 摘要/清除](#)
7. [7. 其他資源](#)

1. 總覽

[無伺服器遷移工作站系列程式碼研究室](#) (自學式實作教學課程) 和[相關影片](#)，旨在協助 [Google Cloud 無伺服器](#)開發人員完成一或多項遷移作業 (主要是從舊版服務遷移)，進而翻新應用程式。這樣做可提高應用程式的可攜性，並提供更多選項和彈性，讓您整合及存取更多 Cloud 產品，並更輕鬆地升級至新版語言。雖然一開始的重點是早期 Cloud 使用者，主要是 [App Engine](#) (標準環境) 開發人員，但本系列涵蓋範圍廣泛，也包括其他無伺服器平台，例如 [Cloud Functions](#) 和 [Cloud Run](#)，或適用於其他平台。

本程式碼研究室 (第 15 模組) 說明如何將 [App Engine blobstore](#) 用量新增至第 0 模組的[範例應用程式](#)。接著，您就可以在第 16 模組中[將該用量遷移至 Cloud Storage](#)。

未使用 Blobstore 嗎？

如果應用程式未使用 App Engine Blobstore 服務，請略過第 15 和 16 個單元，或完成這些程式碼研究室，熟悉 Blobstore 遷移作業。

使用 App Engine Cloud Storage 自訂用戶端程式庫嗎？

如果您的應用程式使用 [Cloud Storage 適用的 App Engine 自訂用戶端程式庫](#) (`GoogleAppEngineCloudStorageClient` 或 `cloudstorage`)，建議您也遷移至 [Cloud Storage](#) 用戶端程式庫。這類遷移作業不在本教學課程的討論範圍內，因此不會說明。[Cloud Storage 遷移頁面](#)提供了一些一般步驟。

在接下來的研究室中

- 新增 App Engine Blobstore API/程式庫的使用方式
- 將使用者上傳內容儲存至 `blobstore` 服務
- 準備遷移至 Cloud Storage 的下一個步驟

軟硬體需求

- 擁有 [Google Cloud Platform 專案](#)，且已啟用 [GCP 帳單帳戶](#)
- Python 基礎技能
- 熟悉常見的 Linux 指令
- 具備[開發及部署 App Engine 應用程式](#)的基本知識
- 可正常運作的 [Module 0 App Engine 應用程式](#) (從存放區取得)

問卷調查

2. 背景

如要從 App Engine Blobstore API 遷移，請將其用法新增至第 0 模組現有的基準 App Engine `ndb` 應用程式。範例應用程式會顯示使用者最近的十次造訪記錄。我們正在修改應用程式，提示使用者上傳與「拜訪」相關的構件 (檔案)。如果使用者不想這麼做，可以選擇「略過」。無論使用者做出什麼決定，下一頁都會轉譯與第 0 模組 (以及本系列中的許多其他模組) 應用程式相同的輸出內容。實作這個 App Engine `blobstore` 整合後，我們就能在下一個 (第 16 個) 程式碼研究室中，將其遷移至 [Cloud Storage](#)。

App Engine 提供 [Django](#) 和 [Jinja2](#) 範本系統的存取權，而這個範例與眾不同之處 (除了新增 Blobstore 存取權之外)，在於它從第 0 個模組的 Django 切換至第 15 個模組的 Jinja2。將網頁架構從 `webapp2` 遷移至 Flask，是 App Engine 應用程式現代化的重要步驟。後者使用 Jinja2 做為預設範本系統，因此我們開始朝這個方向發展，在保留 `webapp2` Blobstore 存取權的同時，實作 Jinja2。由於 Flask 預設使用 Jinja2，因此在第 16 模組中，您不需要對範本進行任何變更。

3. 設定/準備工作

在進入教學課程的主要部分之前，請先設定專案、取得程式碼，並部署基準應用程式，以便開始使用可運作的程式碼。

1. 設定專案

如果您已部署模組 0 應用程式，建議重複使用相同的專案 (和程式碼)。或者，您也可以[建立全新專案](#)，或重複使用其他現有專案。確認專案已啟用 App Engine，且具備有效的帳單帳戶。

2. 取得基準範例應用程式

本程式碼研究室的先決條件之一是擁有可運作的「模組 0」範例應用程式。如果沒有，可以從「模組 0」的「START」資料夾 (下方連結) 取得。本程式碼研究室會逐步說明每個步驟，最後的程式碼會與「Module 15」資料夾中的程式碼類似。

- 開始：[Module 0 資料夾](#) (Python 2)
- 完成：[第 15 堂課資料夾](#) (Python 2)
- [整個存放區](#) (複製或下載 [ZIP 檔案](#))

模組 0 STARTING 檔案的目錄應如下所示：

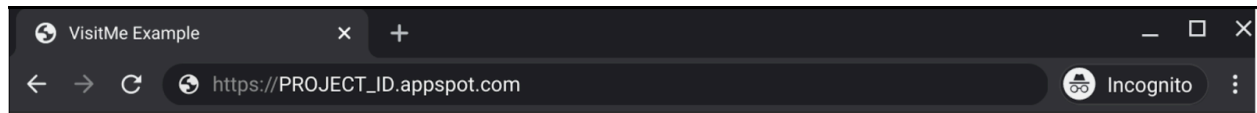
```
$ ls
README.md          index.html
app.yaml           main.py
```

3. (重新) 部署基準應用程式

請立即執行剩餘的準備步驟：

1. 重新熟悉 `gcloud` 指令列工具
2. 使用 `gcloud app deploy` 重新部署範例應用程式
3. 確認應用程式在 App Engine 上順利執行

成功執行上述步驟並確認網頁應用程式正常運作 (輸出內容類似下方範例) 後，即可在應用程式中加入快取功能。



VisitMe example

Last 10 visits

- Tue Mar 30 06:49:20 2021 from 2601:647:4300:5df:2ac6:3fff:fe34:13c7: Mozilla/5.0 (Macintosh; Intel Mac OS X 11_2_2) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/89.0.4389.90 Safari/537.36
 - Sun Mar 28 03:45:17 2021 from 2601:647:4300:5df:2ac6:3fff:fe34:13c7: Mozilla/5.0 (X11; CrOS x86_64 13597.94.0) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/88.0.4324.186 Safari/537.36
 - Sat Mar 27 20:31:55 2021 from 2601:647:4300:5df:2ac6:3fff:fe34:13c7: curl/7.64.0
 - Sat Mar 27 20:23:39 2021 from 2601:647:4300:5df:2ac6:3fff:fe34:13c7: Mozilla/5.0 (X11; CrOS x86_64 13597.94.0) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/88.0.4324.186 Safari/537.36
 - Fri Mar 26 04:09:23 2021 from 2601:647:4300:5df:2ac6:3fff:fe34:13c7: curl/7.64.1
 - Fri Mar 26 04:07:26 2021 from 2601:647:4300:5df:2ac6:3fff:fe34:13c7: curl/7.64.1
 - Sat Mar 20 00:17:54 2021 from 2601:647:4300:5df:2ac6:3fff:fe34:13c7: curl/7.64.1
 - Sat Mar 20 00:02:06 2021 from 2601:647:4300:5df:2ac6:3fff:fe34:13c7: Mozilla/5.0 (Macintosh; Intel Mac OS X 11_2_2) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/89.0.4389.82 Safari/537.36
 - Sat Mar 20 00:00:15 2021 from 2601:647:4300:5df:2ac6:3fff:fe34:13c7: Mozilla/5.0 (Macintosh; Intel Mac OS X 11_2_2) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/89.0.4389.82 Safari/537.36
 - Fri Mar 19 23:28:54 2021 from 127.0.0.1: Mozilla/5.0 (Macintosh; Intel Mac OS X 11_2_2) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/89.0.4389.82 Safari/537.36
-

步驟 4 · 約 3 分鐘

4. 更新設定檔

app.yaml

應用程式設定沒有重大變更，但如先前所述，我們將從 Django 範本 (預設) 遷移至 Jinja2，因此如要切換，使用者應指定 App Engine 伺服器上可用的最新版 Jinja2，方法是將其新增至 `app.yaml` 的內建第三方程式庫部分。

BEFORE:

```
runtime: python27
threadsafe: yes
api_version: 1

handlers:
- url: /*
  script: main.app
```

編輯 `app.yaml` 檔案，新增如下所示的 `libraries` 區段：

修改後：

```
runtime: python27
threadsafe: yes
api_version: 1

handlers:
- url: /*
  script: main.app

libraries:
- name: jinja2
  version: latest
```

需要更新第三方程式庫版本嗎？

上述 `app.yaml` 變更似乎指定了「最新」版本的 Jinja2。不過，這只表示 App Engine 伺服器上可用的最新 Python 2 版本，而非程式庫本身的最新版本。如果需要比[支援的第三方程式庫頁面](#)所列版本更新的版本，請勿按照上述指示將其新增至 `app.yaml`。請改為在 `requirements.txt` 中列出，並執行 `pip install -t lib -r requirements.txt`，為 Python 2 組合/供應實際的最新版 Jinja2 (或任何允許的第三方程式庫)。內建程式庫可免除自行組合的麻煩，但代價是必須 (一律) 提供最新版本。

其他設定檔不需要更新，因此請繼續處理應用程式檔案。

5. 修改應用程式檔案

匯入和 Jinja2 支援

第一組 `main.py` 變更包括新增 Blobstore API 的使用方式，以及將 Django 範本替換為 Jinja2。異動內容如下：

1. `os` 模組的用途是建立 Django 範本的檔案路徑名稱。由於我們改用 Jinja2 處理這項作業，因此不再需要使用 `os` 和 Django 範本轉譯器 `google.appengine.ext.webapp.template`，所以將其移除。
2. 匯入 Blobstore API：`google.appengine.ext.blobstore`
3. 匯入原始 `webapp` 架構中的 Blobstore 處理常式，這些處理常式在 `webapp2` 中無法使用：`google.appengine.ext.webapp.blobstore_handlers`
4. 從 `webapp2_extras` 套件匯入 Jinja2 支援

BEFORE:

```
import os
import webapp2
from google.appengine.ext import ndb
from google.appengine.ext.webapp import template
```

如要實作上述清單中的變更，請將 `main.py` 中的現有匯入部分，替換為下列程式碼片段。

修改後：

```
import webapp2
from webapp2_extras import jinja2
from google.appengine.ext import blobstore, ndb
from google.appengine.ext.webapp import blobstore_handlers
```

匯入後，請新增一些樣板程式碼，支援使用 `webapp2_extras` 文件中定義的 Jinja2。下列程式碼片段會使用 Jinja2 功能包裝標準 `webapp2` 要求處理常式類別，因此請在匯入項目後，將這個程式碼區塊新增至 `main.py`：

```
class BaseHandler(webapp2.RequestHandler):
    'Derived request handler mixing-in Jinja2 support'
    @webapp2.cached_property
    def jinja2(self):
        return jinja2.get_jinja2(app=self.app)

    def render_response(self, _template, **context):
        self.response.write(self.jinja2.render_template(_template, **context))
```

替代 Jinja2 支援

如果您不認識上述 Jinja2 樣板，且預期會看到 [App Engine 文件中的範例](#) (使用 `jinja2.Environment()`)，我們選擇 `webapp2extras` 解決方案是因為 [它提供更完善的 \(I18N 執行緒\) 支援](#)。

新增 Blobstore 支援

與本系列的其他遷移作業不同，這個範例大幅偏離常規，因為我們並未保留範例應用程式的功能或輸出內容，而是幾乎完全改變了使用者體驗。我們更新了應用程式，不再立即登錄新訪客，然後顯示最近十位訪客，而是要求使用者提供檔案構件來登錄訪客。然後，使用者可以上傳相應的檔案，或選取「略過」完全不上傳任何內容。完成這個步驟後，系統會顯示「最近造訪」頁面。

這項變更可讓應用程式使用 Blobstore 服務，在最近造訪的頁面上儲存 (並可能稍後算繪) 該圖片或其他檔案類型。

更新資料模型並實作

我們將儲存更多資料，具體來說，就是更新資料模型，儲存上傳至 Blobstore 的檔案 ID (稱為「BlobKey」)，並新增參照，將該 ID 儲存至 `store_visit()`。由於這項額外資料會在查詢時與其他所有資料一併傳回，因此 `fetch_visits()` 維持不變。

以下是這些更新前後的比較，其中包含 `file_blob`，以及 `ndb.BlobKeyProperty`：

BEFORE:

```
class Visit(ndb.Model):
    'Visit entity registers visitor IP address & timestamp'
    visitor    = ndb.StringProperty()
    timestamp  = ndb.DateTimeProperty(auto_now_add=True)

def store_visit(remote_addr, user_agent):
    'create new Visit entity in Datastore'
    Visit(visitor='{}: {}'.format(remote_addr, user_agent)).put()

def fetch_visits(limit):
    'get most recent visits'
    return Visit.query().order(-Visit.timestamp).fetch(limit)
```

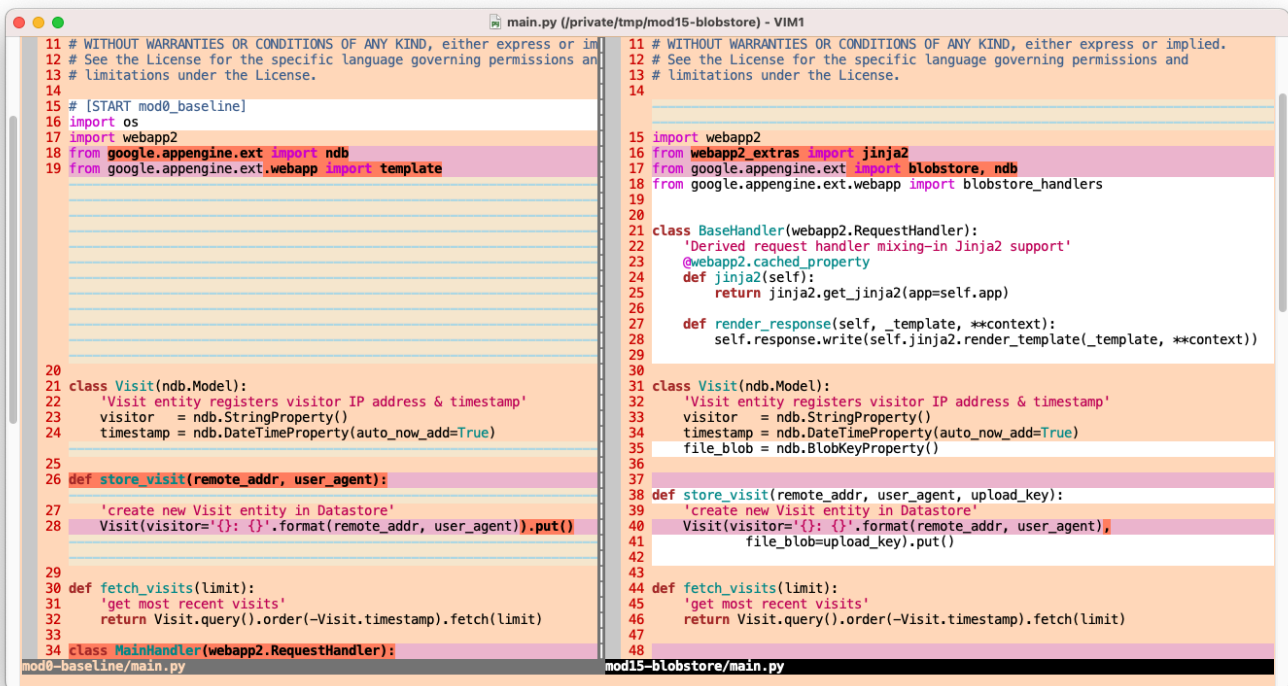
修改後：

```
class Visit(ndb.Model):
    'Visit entity registers visitor IP address & timestamp'
    visitor    = ndb.StringProperty()
    timestamp  = ndb.DateTimeProperty(auto_now_add=True)
    file_blob  = ndb.BlobKeyProperty()

def store_visit(remote_addr, user_agent, upload_key):
    'create new Visit entity in Datastore'
    Visit(visitor='{}: {}'.format(remote_addr, user_agent),
          file_blob=upload_key).put()

def fetch_visits(limit):
    'get most recent visits'
    return Visit.query().order(-Visit.timestamp).fetch(limit)
```

以下是目前為止的變更內容圖示：



支援檔案上傳

功能方面最顯著的變更，就是支援檔案上傳作業，無論是提示使用者上傳檔案、支援「略過」功能，還是轉譯與造訪作業相應的檔案，都包含在內。這些都是圖片的一部分。如要支援檔案上傳功能，必須進行下列變更：

1. 主要處理常式 `GET` 要求不再擷取最近的造訪記錄以供顯示。而是提示使用者上傳。
2. 當使用者提交要上傳的檔案或略過該程序時，表單中的 `POST` 會將控制權傳遞至新的 `UploadHandler`，而該 `UploadHandler` 衍生自 `google.appengine.ext.webapp.blobstore_handlers.BlobstoreUploadHandler`。
3. `UploadHandler` 的 `POST` 方法會執行上傳作業、呼叫 `store_visit()` 註冊造訪，並觸發 HTTP 307 重新導向，將使用者送回「/」，然後...
4. 主要處理常式的 `POST` 方法會查詢 (透過 `fetch_visits()`) 並顯示最近的造訪記錄。如果使用者選取「略過」，系統不會上傳任何檔案，但仍會記錄造訪，並進行相同的重新導向。
5. 「最近的造訪記錄」顯示畫面會向使用者顯示新欄位，如果可上傳檔案，則顯示超連結的「查看」，否則顯示「無」。這些變更會在 HTML 範本中實現，並新增上傳表單 (詳情即將推出)。
6. 如果使用者點選任何已上傳影片的造訪記錄「查看」連結，系統會向衍生自 `google.appengine.ext.webapp.blobstore_handlers.BlobstoreDownloadHandler` 的新 `ViewBlobHandler` 發出 `GET` 要求，並視情況轉譯檔案 (如果是圖片，則在瀏覽器中轉譯，如果瀏覽器不支援，則提示下載)，或在找不到檔案時傳回 HTTP 404 錯誤。
7. 除了新的處理常式類別配對，以及將流量傳送至這些類別的新路徑配對之外，主要處理常式還需要新的 `POST` 方法，才能接收上述 307 重新導向。

在這些更新之前，Module 0 應用程式只會提供具有 `GET` 方法的主要處理常式和單一路徑：

BEFORE:

```

class MainHandler(webapp2.RequestHandler):
    'main application (GET) handler'
    def get(self):
        store_visit(self.request.remote_addr, self.request.user_agent)
        visits = fetch_visits(10)
        tpl = os.path.join(os.path.dirname(__file__), 'index.html')
        self.response.out.write(template.render(tpl, {'visits': visits}))

app = webapp2.WSGIApplication([
    ('/', MainHandler),
], debug=True)

```

實作這些更新後，現在有三個處理常式：1) 含有 `POST` 方法的上傳處理常式、2) 含有 `GET` 方法的「檢視 Blob」下載處理常式，以及 3) 含有 `GET` 和 `POST` 方法的主要處理常式。進行這些變更後，應用程式的其餘部分應如下所示。

修改後：

```

class UploadHandler(blobstore_handlers.BlobstoreUploadHandler):
    'Upload blob (POST) handler'
    def post(self):
        uploads = self.get_uploads()
        blob_id = uploads[0].key() if uploads else None
        store_visit(self.request.remote_addr, self.request.user_agent, blob_id)
        self.redirect('/', code=307)

class ViewBlobHandler(blobstore_handlers.BlobstoreDownloadHandler):
    'view uploaded blob (GET) handler'
    def get(self, blob_key):
        self.send_blob(blob_key) if blobstore.get(blob_key) else self.error(404)

class MainHandler(BaseHandler):
    'main application (GET/POST) handler'
    def get(self):
        self.render_response('index.html',
                             upload_url=blobstore.create_upload_url('/upload'))

    def post(self):
        visits = fetch_visits(10)
        self.render_response('index.html', visits=visits)

app = webapp2.WSGIApplication([
    ('/', MainHandler),
    ('/upload', UploadHandler),
    ('/view/([^\s]+)?', ViewBlobHandler),
], debug=True)

```

我們剛新增的程式碼中有幾個重要呼叫：

- 在 `MainHandler.get` 中，有對 `blobstore.create_upload_url` 的呼叫。這項呼叫會產生表單 `POST` 的網址，並呼叫上傳處理常式，將檔案傳送至 Blobstore。

- 在 `UploadHandler.post` 中，有對 `blobstore_handlers.BlobstoreUploadHandler.get_uploads` 的呼叫。這才是真正的魔法，可將檔案放入 Blobstore，並傳回該檔案的專屬永久 ID，也就是 `BlobKey`。
- 在 `ViewBlobHandler.get` 中，使用檔案的 `BlobKey` 呼叫 `blobstore_handlers.BlobstoreDownloadHandler.send` 會導致系統擷取檔案，並將其轉送至使用者的瀏覽器

這些呼叫代表存取應用程式新增功能的大宗作業。以下是 `main.py` 的第二組也是最後一組變更的圖像表示法：

```

30 def fetch_visits(limit):
31     'get most recent visits'
32     return Visit.query().order(-Visit.timestamp).fetch(limit)
33
34 class MainHandler(webapp2.RequestHandler):
35     'main application (GET) handler'
36
37     def get(self):
38         store_visit(self.request.remote_addr, self.request.user_agent)
39
40         visits = fetch_visits(10)
41         tpl = os.path.join(os.path.dirname(__file__), 'index.html')
42         self.response.out.write(template.render(tpl, {'visits': visits}))
43
44 app = webapp2.WSGIApplication([
45     ('/', MainHandler),
46 ], debug=True)
47 # [END mod0_baseline]
48
49 def fetch_visits(limit):
50     'get most recent visits'
51     return Visit.query().order(-Visit.timestamp).fetch(limit)
52
53 class UploadHandler(blobstore_handlers.BlobstoreUploadHandler):
54     'Upload blob (POST) handler'
55     def post(self):
56         uploads = self.get_uploads()
57         blob_id = uploads[0].key() if uploads else None
58         store_visit(self.request.remote_addr, self.request.user_agent, blob_id)
59         self.redirect('/', code=307)
60
61 class ViewBlobHandler(blobstore_handlers.BlobstoreDownloadHandler):
62     'view uploaded blob (GET) handler'
63     def get(self, blob_key):
64         self.send_blob(blob_key) if blobstore.get(blob_key) else self.error(404)
65
66 class MainHandler(BaseHandler):
67     'main application (GET/POST) handler'
68     def get(self):
69         self.render_response('index.html',
70                             upload_url=blobstore.create_upload_url('/upload'))
71     def post(self):
72         visits = fetch_visits(10)
73         self.render_response('index.html', visits=visits)
74
75 app = webapp2.WSGIApplication([
76     ('/', MainHandler),
77     ('/upload', UploadHandler),
78     ('/view/([^/]+)?', ViewBlobHandler),
79 ], debug=True)

```

更新 HTML 範本

主要應用程式的部分更新會影響應用程式的使用者介面 (UI)，因此網頁範本也必須進行相應變更，實際上需要變更兩處：

1. 您必須使用檔案上傳表單，其中包含 3 個輸入元素：檔案，以及一組分別用於上傳檔案和略過的提交按鈕。
2. 為有對應檔案上傳作業的造訪記錄新增「查看」連結，否則新增「無」，藉此更新最近造訪記錄的輸出內容。

BEFORE:

```
<!doctype html>
<html>
<head>
<title>VisitMe Example</title>
<body>

<h1>VisitMe example</h1>
<h3>Last 10 visits</h3>
<ul>
{% for visit in visits %}
    <li>{{ visit.timestamp.ctime }} from {{ visit.visitor }}</li>
{% endfor %}
</ul>

</body>
</html>
```

實作上述清單中的變更，即可組成更新後的範本：

修改後：

```

<!doctype html>
<html>
<head>
<title>VisitMe Example</title>
<body>

<h1>VisitMe example</h1>
{% if upload_url %}

<h3>Welcome... upload a file? (optional)</h3>
<form action="{{ upload_url }}" method="POST" enctype="multipart/form-data">
    <input type="file" name="file"><p></p>
    <input type="submit"> <input type="submit" value="Skip">
</form>

{% else %}

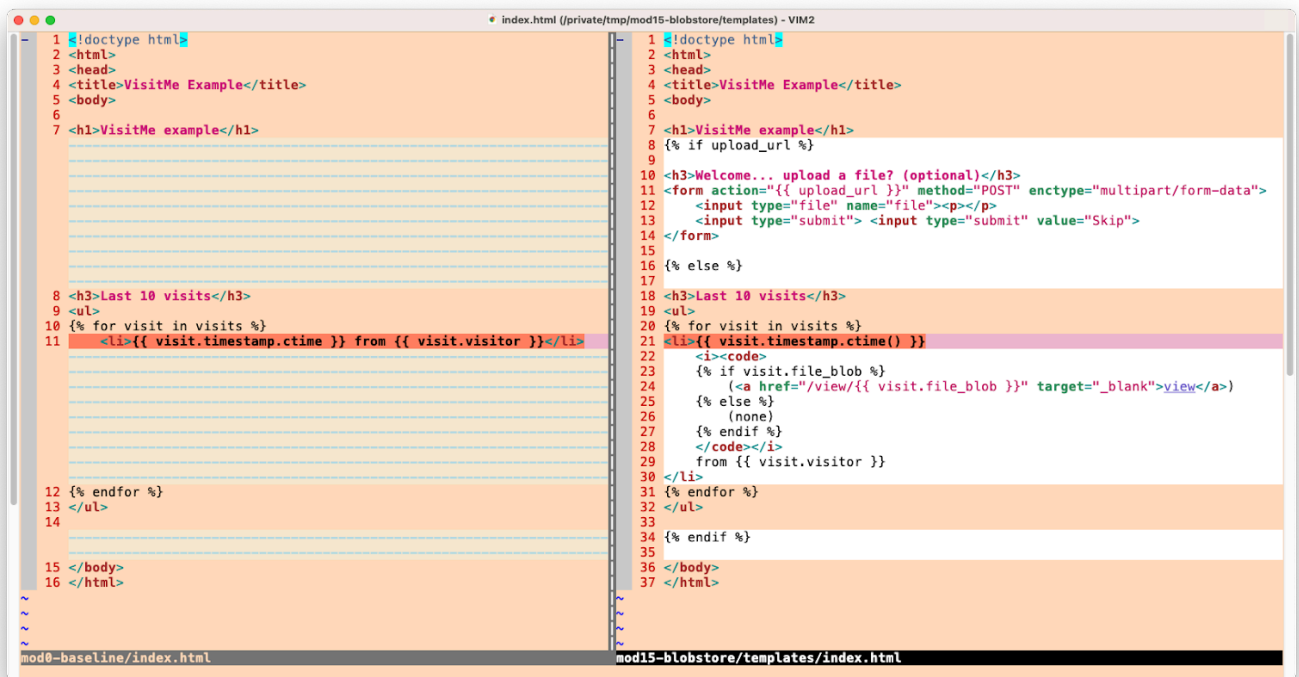
<h3>Last 10 visits</h3>
<ul>
{% for visit in visits %}
<li>{{ visit.timestamp.ctime() }}
    <i><code>
        {% if visit.file_blob %}
            (<a href="/view/{{ visit.file_blob }}" target="_blank">view</a>)
        {% else %}
            (none)
        {% endif %}
    </code></i>
    from {{ visit.visitor }}
</li>
{% endfor %}
</ul>

{% endif %}

</body>
</html>

```

下圖說明 `index.html` 的必要更新：

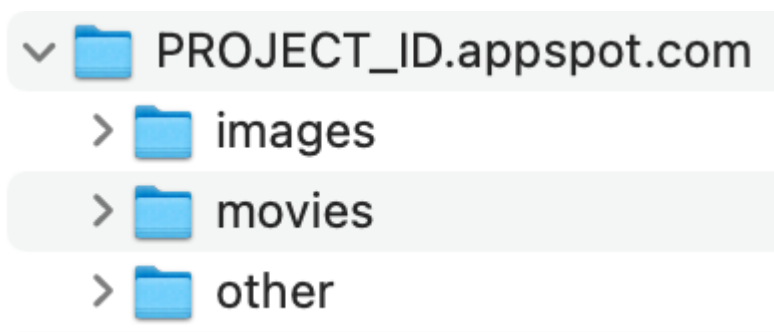


最後一項變更是 Jinja2 偏好使用 `templates` 資料夾中的範本，因此請建立該資料夾，然後將 `index.html` 移至該資料夾中。完成最後這項作業後，您就已完成所有必要變更，可將 Blobstore 的使用新增至 Module 0 範例應用程式。

(選用) Cloud Storage 「強化」 功能

Blobstore 儲存空間最終演變為 Cloud Storage 本身。也就是說，Blobstore 上傳作業會顯示在 Cloud 控制台 (具體來說是 Cloud Storage 瀏覽器)。問題在於地點。答案是 App Engine 應用程式的預設 Cloud Storage 值區。其名稱為 App Engine 應用程式的完整網域名稱，即 `PROJECT_ID.appspot.com`。因為所有專案 ID 都是不重複的，所以非常方便，對吧？

範例應用程式更新後，上傳的檔案會放入該 bucket，但開發人員可以選擇更具體的位置。預設值區可透過 `google.appengine.api.app_identity.get_default_gcs_bucket_name()` 以程式輔助方式存取，因此如要存取這個值 (例如用來做為前置字元，整理上傳的檔案)，就必須匯入新值。舉例來說，依檔案類型排序：



舉例來說，如要為圖片實作類似功能，您會使用下列程式碼，以及檢查檔案類型以挑選所需 bucket 名稱的程式碼：

```
ROOT_BUCKET = app_identity.get_default_gcs_bucket_name()
IMAGE_BUCKET = '%s/%s' % (ROOT_BUCKET, 'images')
```

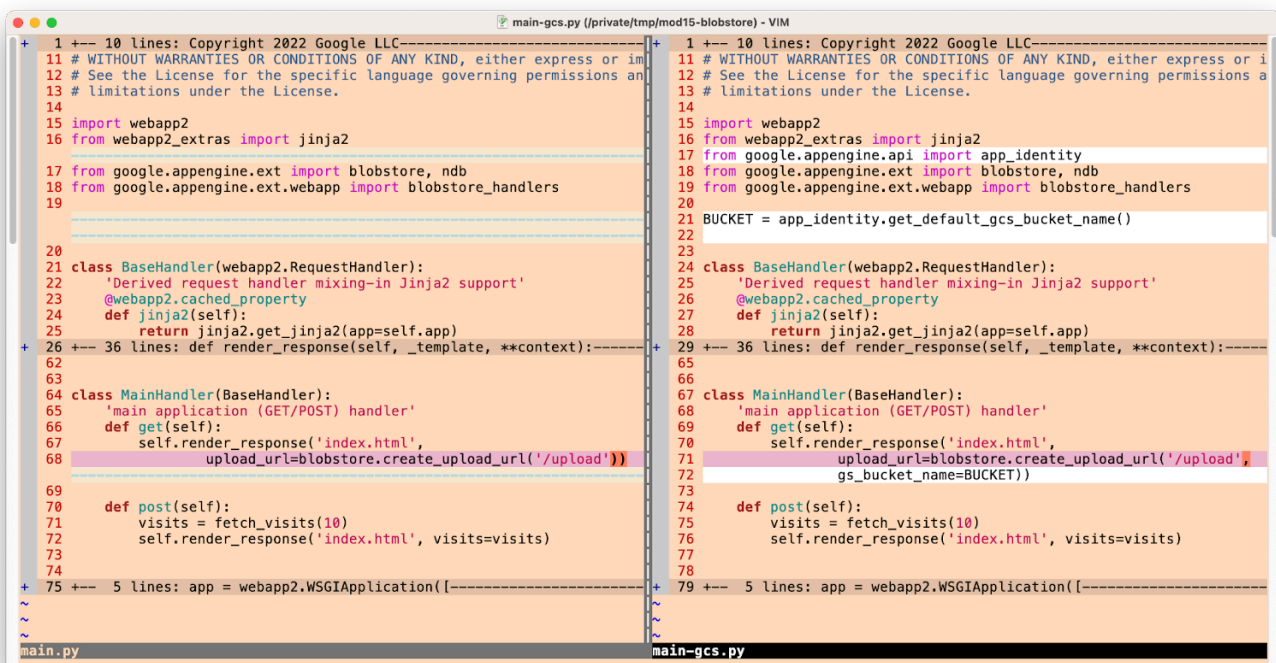
您也可以使用 Python 標準程式庫 `imghdr` 模組等工具驗證上傳的圖片，確認圖片類型。最後，您可能需要限制上傳大小，以防惡意行為人濫用。

假設上述所有步驟都已完成，如何更新應用程式，支援指定上傳檔案的儲存位置？關鍵在於調整 `MainHandler.get` 中的 `blobstore.create_upload_url` 呼叫，加入 `gs_bucket_name` 參數，指定要將上傳內容儲存在 Cloud Storage 中的

位置，如下所示：

```
blobstore.create_upload_url('/upload', gs_bucket_name=IMAGE_BUCKET))
```

這項更新為選用項目，可指定上傳檔案的儲存位置，因此不屬於存放區中的 `main.py` 檔案。而是提供名為 `main-gcs.py` 的替代方案供您參考。程式碼會將上傳內容儲存在「根」值區 (`PROJECT_ID.appspot.com`)，而非使用獨立的「資料夾」，就像 `main.py` 一樣，但如果您要將範例衍生為本節中提示的內容，程式碼會提供所需架構。`main-gcs.py` 下圖說明 `main.py` 和 `main-gcs.py` 之間的「差異」。



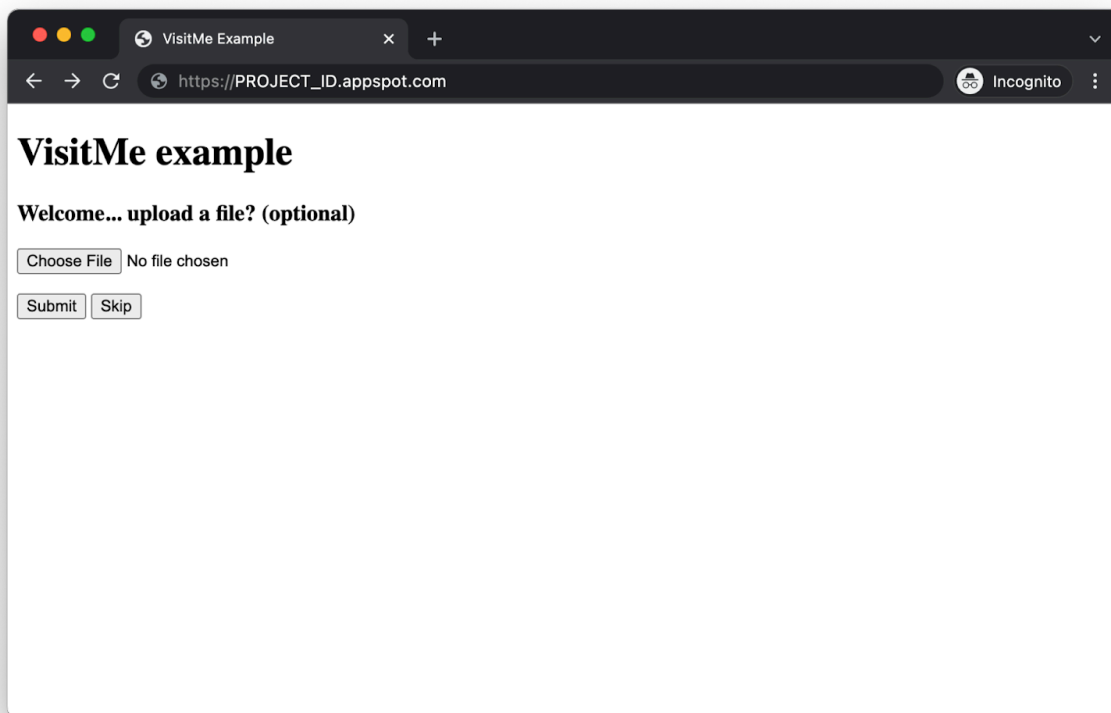
步驟 6 · 約 5 分鐘

6. 摘要/清除

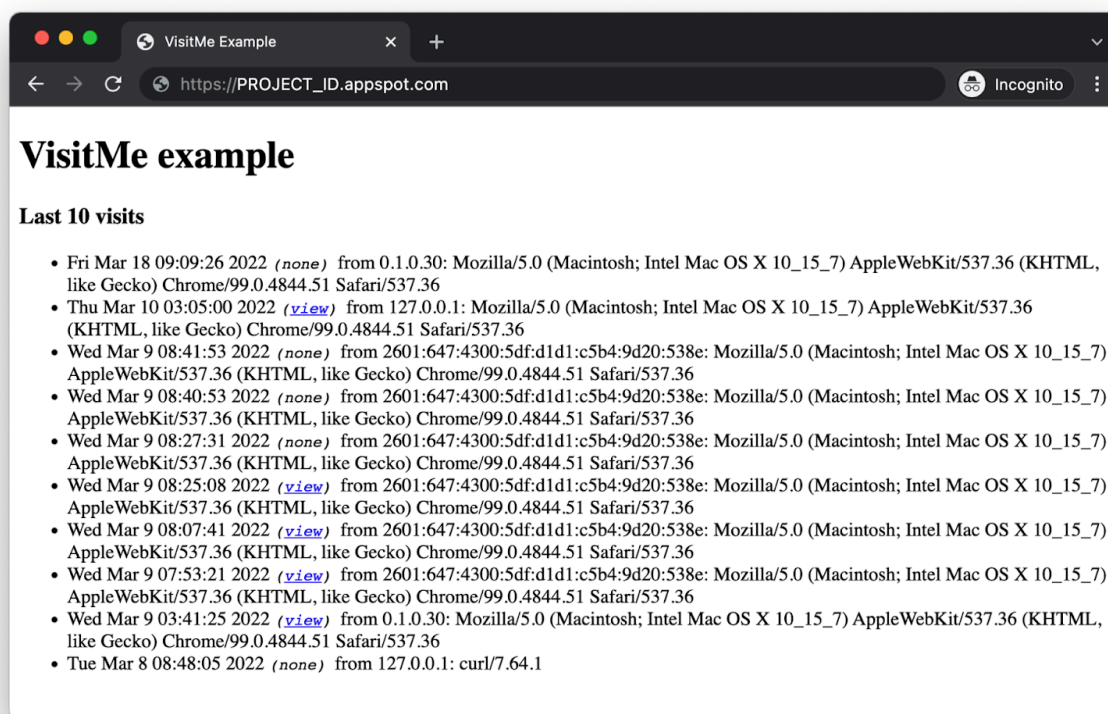
本節將部署應用程式，並驗證應用程式是否正常運作，以及是否反映在輸出內容中，為本程式碼研究室畫下句點。應用程式驗證完成後，請執行任何清理步驟，並考慮後續步驟。

部署及驗證應用程式

使用 `gcloud app deploy` 重新部署應用程式，並確認應用程式運作方式與廣告宣傳內容一致，且使用者體驗 (UX) 與模組 0 應用程式不同。現在應用程式中有兩個不同的畫面，第一個是檔案上傳表單提示：



使用者可以上傳檔案並按一下「提交」，也可以按一下「略過」不上傳任何內容。無論是哪種情況，結果都是最新的造訪畫面，現在造訪時間戳記和訪客資訊之間會顯示「查看」連結或「無」：



恭喜您完成本程式碼研究室，並在 Module 0 範例應用程式中新增 App Engine Blobstore 的使用方式。您的程式碼現在應該與 [FINISH \(Module 15\) 資料夾](#) 中的程式碼相符。該資料夾中也有替代的 `main-gcs.py`。

清除所用資源

一般

如果暫時不需要使用，建議[停用 App Engine 應用程式](#)，以免產生費用。不過，如果您想進一步測試或實驗，App Engine 平台提供[免付費配額](#)，只要不超出該用量層級，就不會產生費用。這是指運算費用，但您可能也需要支付相關 App Engine 服務的費用，因此請參閱[其定價頁面](#)瞭解詳情。如果這項遷移作業涉及其他雲端服務，則這些服務會另外計費。無論是哪種情況，請視需要參閱下方的「本程式碼研究室專用」一節。

為求完整揭露，部署至 App Engine 等 Google Cloud 無伺服器運算平台時，會產生[少量建構和儲存空間費用](#)。[Cloud Build](#) 和 [Cloud Storage](#) 都有各自的免費配額。儲存該圖片會耗用部分配額。不過，你所在的區域可能沒有這類免付費層級，因此請留意儲存空間用量，盡量減少潛在費用。您應審查的特定 Cloud Storage「資料夾」包括：

- `console.cloud.google.com/storage/browser/LOC.artifacts.PROJECT_ID.appspot.com/containers/images`
- `console.cloud.google.com/storage/browser/staging.PROJECT_ID.appspot.com`
- 上述儲存空間連結取決於您的 `PROJECT_ID` 和 `*LOC*ation`，例如，如果您的應用程式託管於美國，則為「us」。

另一方面，如果您不打算繼續使用這個應用程式或其他相關的遷移 Codelab，並想完全刪除所有內容，請[關閉專案](#)。

本程式碼研究室專用

下列服務是本程式碼研究室的專屬服務。詳情請參閱各項產品的說明文件：

- App Engine Blobstore 服務適用於[儲存資料配額和限制](#)，因此請一併參閱[舊版套裝服務的價格頁面](#)。
- [App Engine Datastore](#) 服務是由 [Cloud Datastore](#) (Cloud Firestore Datastore 模式) 提供，這項服務也有免費方案；詳情請參閱[定價頁面](#)。

後續步驟

第 16 模組將介紹下一個可考慮的邏輯遷移作業，說明開發人員如何從 App Engine Blobstore 服務遷移至 Cloud Storage 用戶端程式庫。升級的好處包括：存取更多 Cloud Storage 功能，以及熟悉適用於 App Engine 以外應用程式的用戶端程式庫 (無論是在 Google Cloud、其他雲端或地端部署)。如果您認為不需要 Cloud Storage 的所有功能，或擔心對費用造成影響，可以繼續使用 App Engine Blobstore。

除了第 16 節以外，還有許多其他可能的遷移作業，例如 Cloud NDB 和 Cloud Datastore、Cloud Tasks 或 Cloud Memorystore。此外，您也可以將產品遷移至 Cloud Run 和 Cloud Functions。[遷移存放區](#)提供所有程式碼範例，並連結至所有可用的程式碼研究室和影片，同時也提供指南，說明要考慮哪些遷移作業，以及任何相關的遷移「順序」。

7. 其他資源

程式碼研究室問題/意見回饋

如果發現本程式碼研究室有任何問題，請先搜尋問題，再提出回報。搜尋及建立新問題的連結：

- [搜尋問題](#)
- [回報新問題/提供意見](#)

遷移資源

您可以在下表中找到模組 0 (START) 和模組 15 (FINISH) 的存放區資料夾連結。您也可以從[所有 App Engine Codelab 遷移作業的存放區](#)存取這些範例，並複製或下載 ZIP 檔案。

Codelab	Python 2	Python 3
單元 0	code	不適用
單元 15 (本程式碼研究室)	code	不適用

線上資源

以下是可能與本教學課程相關的線上資源：

App Engine

- [App Engine Blobstore 服務](#)
- [App Engine 儲存資料配額與限制](#)
- [App Engine 說明文件](#)
- [Python 2 App Engine \(標準環境\) 執行階段](#)
- [在 Python 2 App Engine 上使用 App Engine 內建程式庫](#)
- [App Engine 定價和配額資訊](#)
- [推出第二代 App Engine 平台 \(2018 年\)](#)
- [比較第一代和第二代平台](#)
- [對舊版執行階段的長期支援](#)
- [說明文件遷移範例存放區](#)
- [社群貢獻的遷移範例存放區](#)

Google Cloud

- [在 Google Cloud Platform 上執行 Python](#)
- [Google Cloud Python 用戶端程式庫](#)
- [Google Cloud 「永久免費」 方案](#)
- [Google Cloud SDK \(gcloud 指令列工具\)](#)
- [所有 Google Cloud 說明文件](#)

Python

- [Django](#) 和 [Jinja2](#) 範本系統
- [webapp2](#) 網頁架構
- [webapp2](#) 說明文件
- [webapp2_extras](#) 連結
- [webapp2_extras](#) Jinja2 說明文件

影片

- [無伺服器遷移工作站](#)
- [無伺服器 Expeditions](#)
- 訂閱 [Google Cloud Tech](#)
- 訂閱 [Google Developers](#)

授權

這項內容採用的授權為 [Creative Commons 姓名標示 2.0 通用授權](#)。
